



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 第二次实验

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 9 月 30 日

实 验 报 告

【实验目的】

1.继续熟悉 Eclipse 的使用

【实验要求】

一、编写一个简单的 Applet 程序

二、在 Eclipse 平台中实现下列简单的程序

【实验具体内容】

一、编写一个简单的 Applet 程序

(1) 实验源程序

```
package cn;
import java.applet.*;
import java.awt.*;
public class MyFirstApplet extends Applet{
    public void Paint(Graphics g)
    {
        g.setColor(Color.blue);
        g.drawString("这是一个 Java Applet 程序。",40,40);
        g.setColor(Color.red);
        g.setFont(new Font("宋体",Font.BOLD,36));
        g.drawString("我改变了字体。",80,80);
    }
}
```

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

1) 利用 eclipse 编写代码，并运行代码。

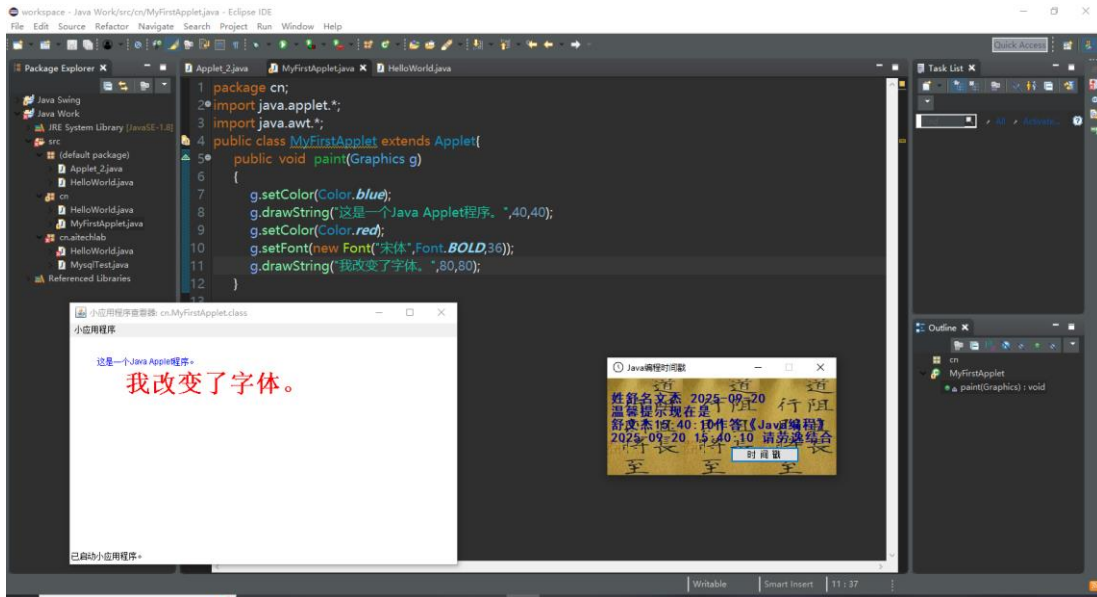


图 1 运行结果截图

可以发现，我们成功实现该代码，于是我们开始进行下一步的试验。

2) 试验各种修饰词的作用

a. 不使用 public 修饰主类

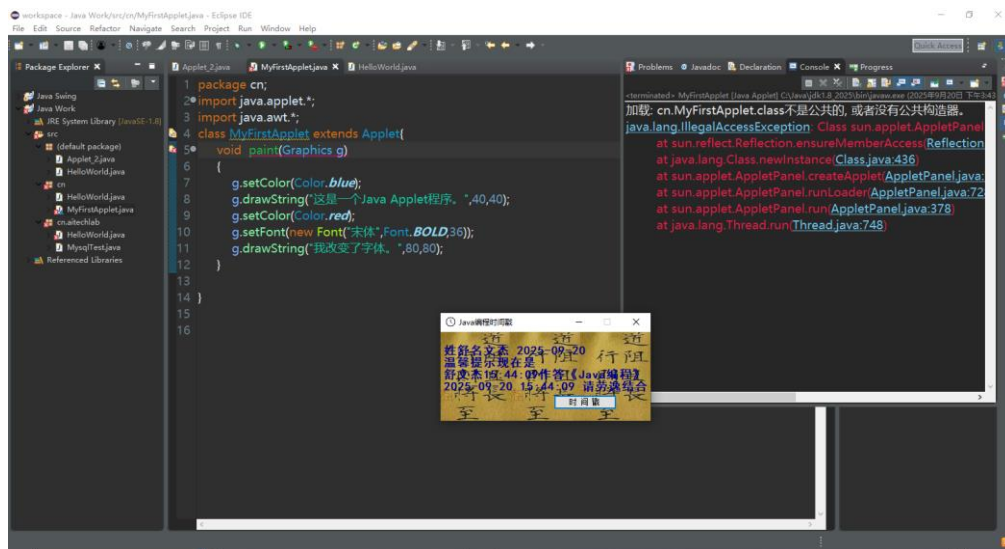


图 2 不使用 public 修饰主类

当我们去掉 `public` 修饰后，我们的编译器便开始报错，编译器提醒我们没有对应的构造器进行构造，这是为什么呢，我查阅了相关资料，发现当我们没有对主类进行修饰时，其默认修饰词为 `private` 与 `protected`，这会使其他类访问不到该类。同时，我们的 Applet 函数会默认被调用 `AppletViewer` 调用，因此会报错找不到构

造器。

b. 将 paint 方法误写成 Paint

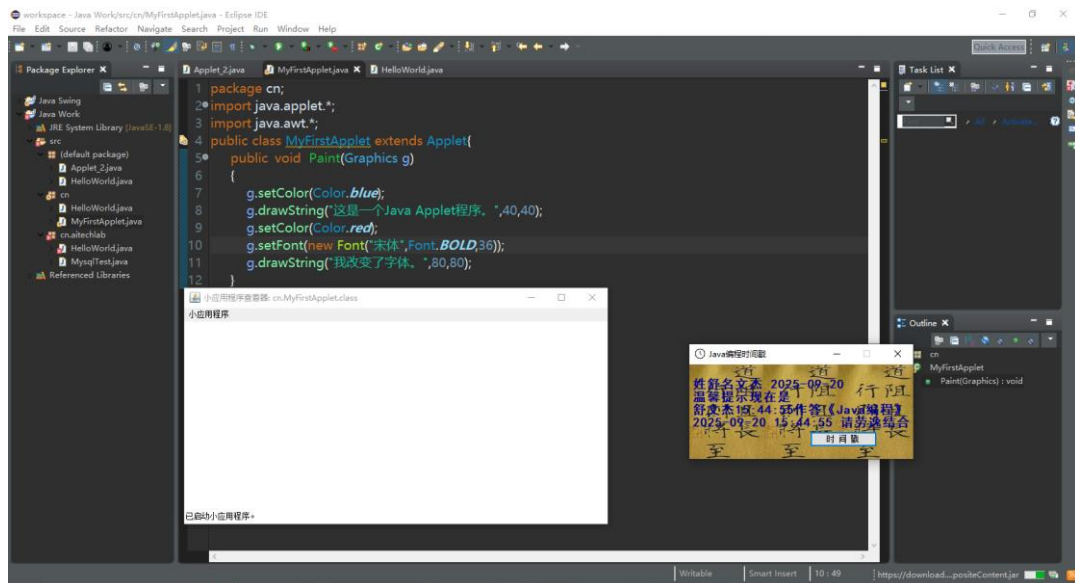


图 3 将 paint 方法误写成 Paint

当我们将 paint 方法误写成 Paint 时，由于 Java 是大小写敏感的，因此 JVM 会认为 Paint 是一个自定义方法，而不是重写 Applet 的绘图方法，因此运行时不会自动调用这个方法，也就无法显示我绘制的内容。

1、实验后的练习：

1) 程序中的主类如果不用 public 修饰，编译能通过吗？

编译不能通过，外部方法无法访问构造的主类。

2) 程序中的主类如果不用 public 修饰，程序能正确运行吗？

不能正确运行，编译无法通过。

3) 程序将 paint 方法误写成 Paint，编译能通过么？

能通过，JVM 会将 Paint 识别成一个我们自定义的方法，同时又不会像 C 语言那样 include 函数，因此在编译层面没有问题

4) 程序将 paint 方法误写成 Paint，运行时能看到有关的输出信息吗？

不能，因为实际我们并没有定义这样一个方法，这是一个空的方法，无法正确显示。

二、在 Eclipse 平台中实现下列简单的程序：

1、1—1000 里面包含的所有的质数，用电脑求它们的和是多少？

(1) 实验源程序

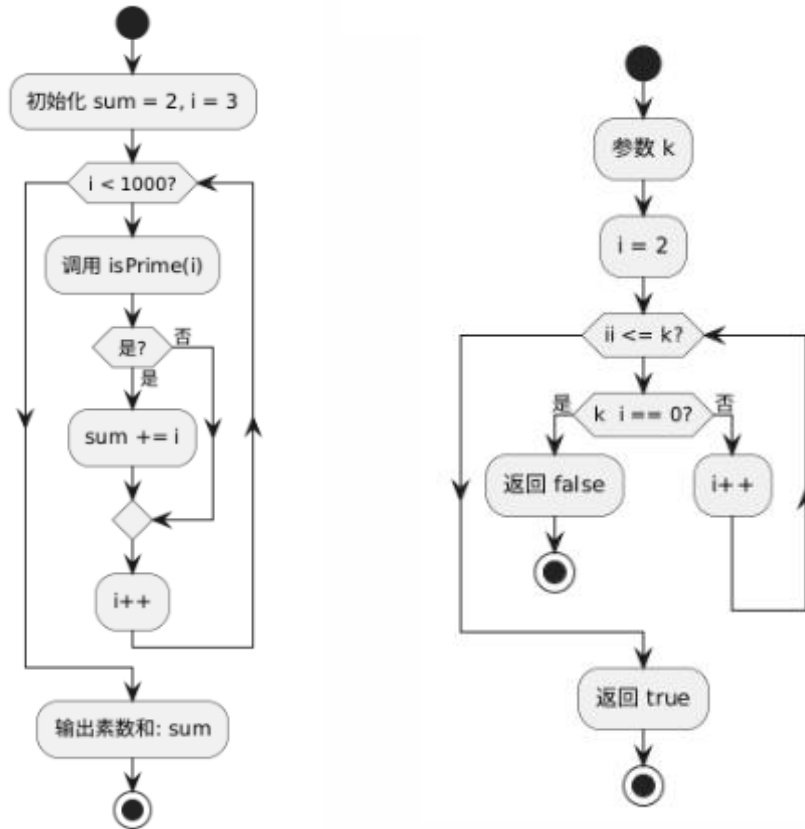
```
package cn;

public class SumPrimeNumber {
    static boolean isPrime(int k) {
        int i;
        for(i = 2; i*i <= k; i++) {
            if(k%i == 0)
            {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int i, sum = 2;

        for(i = 3; i < 1000; i++)
        {
            if(isPrime(i))
            {
                sum += i;
            }
        }
        System.out.printf("所有素数和为: %d", sum);
    }
}
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

思路：对于该程序，我们只需要进行枚举并判断素数的后累加即可，比较基础。

1) 编写 java 源程序并运行

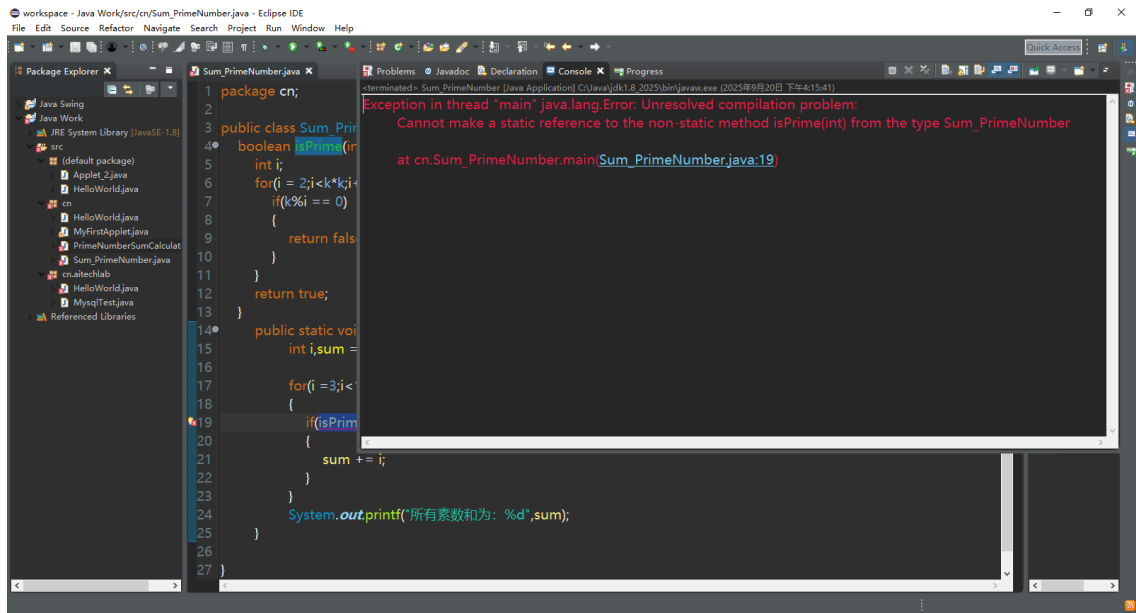


图 4 运行结果截图

？，为什么会发生编译错误，仔细观察后我才发现我的方法前面少了 `static` 修饰，于是我查阅了相关资料来了解 `static` 的作用。原来我在静态的 `main` 方法中调用方法只能调用同为静态的方法，若无 `static` 修饰，方法默认是动态的，就无法被静态的 `main` 调用。

3) 修正语法错误后，我重新进行实验

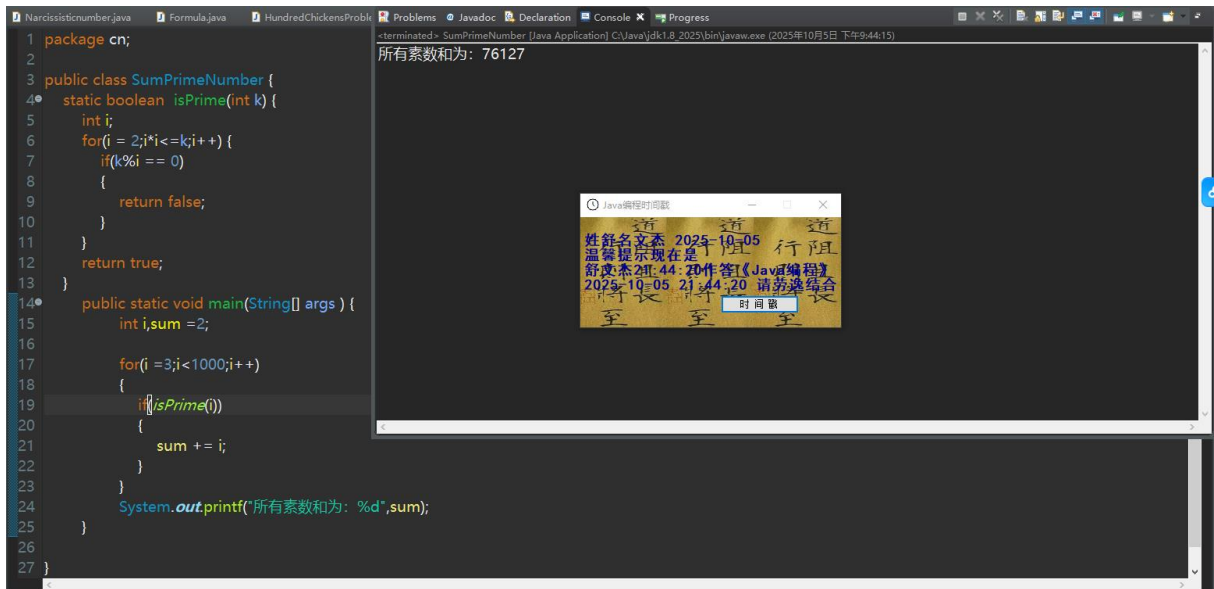


图 5 执行结果截图

可以发现，我们的结果是正确的

2、首先采用递归算法实现阶乘的计算，然后利用循环，并调用此函数实现求和
求出 $1! + 2! + 3! + \dots + 10!$ 的阶乘之和。

(1) 实验源程序

```
package cn;

public class Faction {

    static int factorial(int n) {
        if(n == 1) {
            return 1;
        }
        else {
            return n *factorial(n-1);
        }
    }

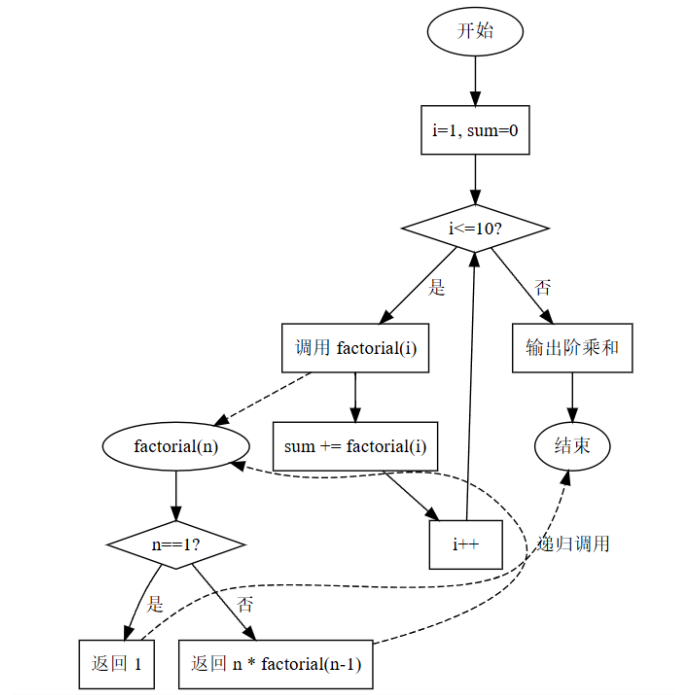
    public static void main(String[] args ) {
        int i =1;
        int sum = 0;
        for (i=1;i<=10;i++)
```

```

    {
        sum+= factorial(i);
    }

    System.out.printf("1~10 的阶乘和为: %d",sum);
}
}
    
```

(2) 实验流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验思路：我们对于阶乘可以归纳出其递归式，即 $n=1$ 时返回 1，若不是一即返回 $n*f(n-1)$ ，在继续枚举前 n 项阶乘后累加即可。

1) 编写 java 源程序并运行

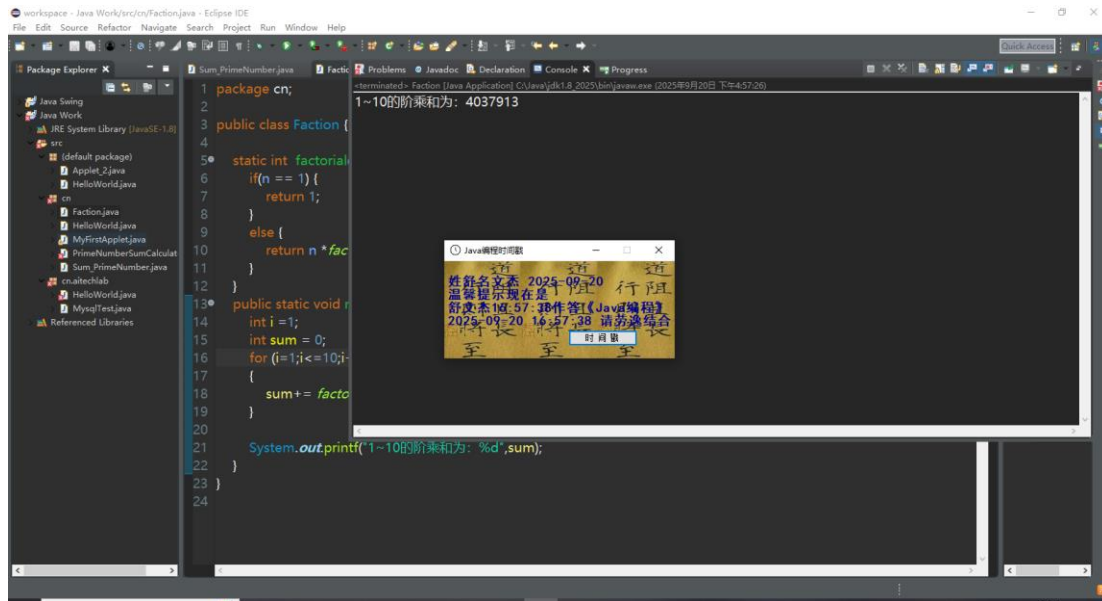


图 6 程序运行结果截图

可以发现，我们的结果是正确的，运行结果符合预期，实验成功。

3、尝试用 applet 编写程序实现摄氏温度和华氏温度的转换。

(1) 实验源程序

```
package cn;

import java.applet.Applet;
import java.awt.Label;
import java.awt.TextField;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.Button;

public class Applet_Temperature extends Applet implements ActionListener {
    private static final long serialVersionUID = 1L;
    TextField temperature1, temperature2;
    Label lab1, lab2;
    Button convertButton;
    public void init() {
        lab1 = new Label("摄氏温度: ");
        lab2 = new Label("华氏温度: ");
        temperature1 = new TextField(10);
        temperature2 = new TextField(10);
        convertButton = new Button("转换");
        add(lab1);
        add(temperature1);
        add(lab2);
    }
}
```

```

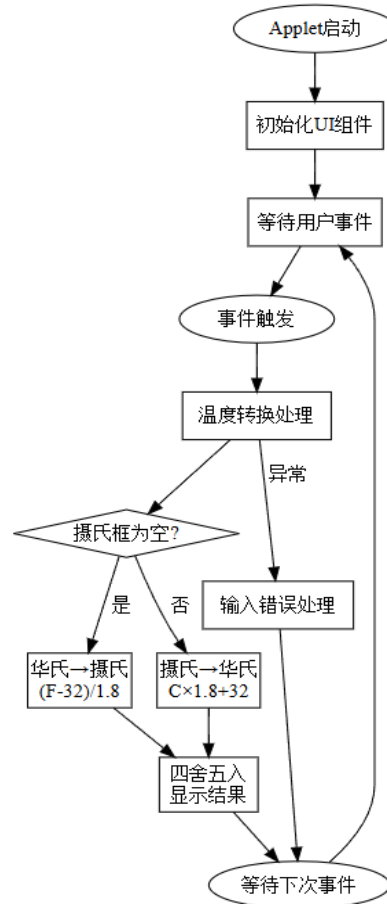
        add(temperature2);
        add(convertButton);
        temperature1.addActionListener(this);
        temperature2.addActionListener(this);
        convertButton.addActionListener(this);
    }

    public void actionPerformed(ActionEvent evt) {
        convertTemperature();
    }

    public void convertTemperature() {
        try {
            if (temperature1.getText().equals("")){
                String fah = temperature2.getText();
                Double fah1 = Double.valueOf(fah).doubleValue();
                Double cel2 = (fah1 - 32) / 1.8;
                float cel3 = (float) (Math.round(cel2 * 100) / 100);
                temperature1.setText(cel3 + "°C");
            }
            else {
                String cel = temperature1.getText();
                Double cel1 = Double.valueOf(cel).doubleValue();
                Double fah2 = 1.8 * cel1 + 32;
                float fah3 = (float) (Math.round(fah2 * 100) / 100);
                temperature2.setText(fah3 + "°F"); // 显示华氏温度结果
            }
        }
        catch (NumberFormatException e) {
            temperature1.setText( "输入错误");
            temperature2.setText( "");
        }
    }
}

```

(2) 实验流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验思路：对于该程序，我们需要先学习老师所给出的框架。从中，我们学习到了组成 Applet 小程序的一些基本的类，同时，我发现可以加入 button 类，并且监听点击按钮的事件，触发后就进行温度的转换。

(1) 编写 java 程序，并进行运行。

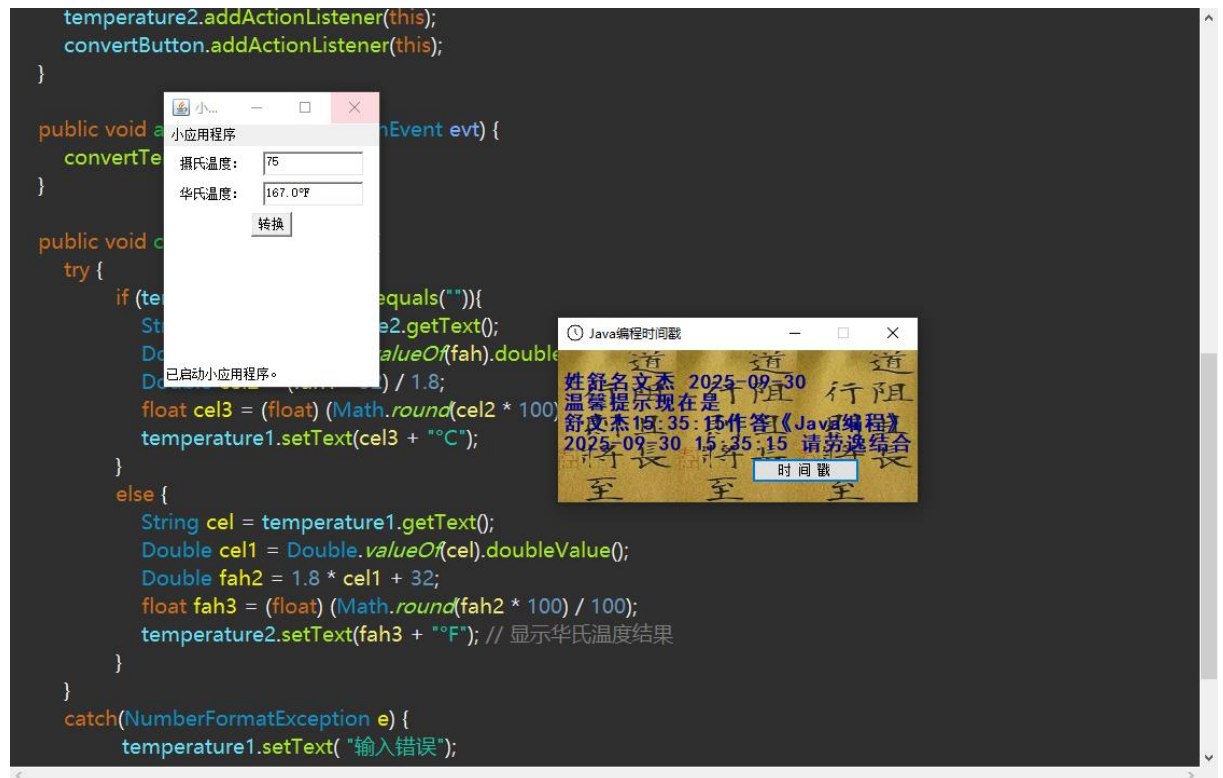


图 7 运行结果截图

可以发现，我们的运行结果符合预期，那么我们进一步进行异常处理。查询相关资料，我们发现，大家常用 `try.....catch` 语句进行异常处理，我们的期望是当用户输入非数字量后，我们便提醒用户输入错误。

2) 加入异常处理后，我们重新进行实验



图 8 异常处理结果截图

我们的运行结果符合预期，实验成功

4、利用 if 语句实现阶梯电费的问题：用电量 240 度以下（包含 240 度）每度电 0.55 元，用电量在 240–540 度之间，超出 240 度电的部分按 0.70 元计算，用电量超过 540 度电，超出 540 度电的部分按照 0.95 元计算。

(1) 实验源程序

```
package cn;

import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

public class CaculateFee extends JFrame implements ActionListener {
    private static final long serialVersionUID = 1072582448511199623L;
    JPanel jp = null;
    JButton convertButton;
    TextField getConsumption, fee;
    Label label1, label2, label3;

    public static void main(String[] args) {
        new CaculateFee();
    }

    public CaculateFee() {
        init();
        this.setSize(300, 300);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    void init() {
        jp = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 15));
        label1 = new Label("请输入所消耗的电量（单位：度）");
        getConsumption = new TextField(30);
        fee = new TextField(30);
        fee.setEditable(false);
        convertButton = new JButton("计算电费");

        jp.add(label1);
        jp.add(getConsumption);
        jp.add(new Label("计算结果（元）："));
        jp.add(fee);
    }
}
```

```

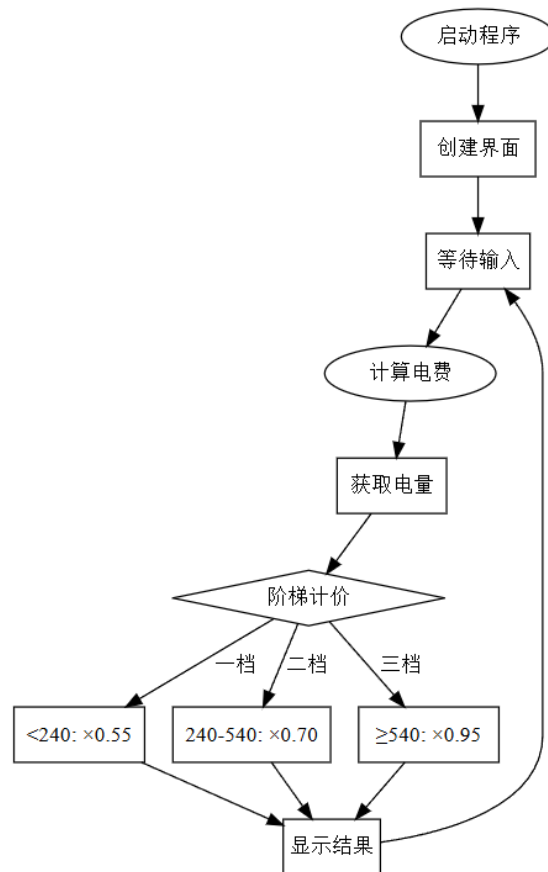
        this.add(jp, BorderLayout.CENTER);
        this.add(convertButton, BorderLayout.SOUTH);
        convertButton.addActionListener(this);
    }

    public void actionPerformed(ActionEvent evt) {
        caculateFee();
    }

    public void caculateFee() {
        try {
            String fah = getConsumption.getText();
            Double fah1 = Double.valueOf(fah).doubleValue();
            Double fee1;
            if(fah1<0) {
                fee.setText("电量不能为负数");
                return;
            }
            else if(fah1 < 240) {
                fee1 = fah1 *0.55;
            }
            else if( fah1 < 540) {
                fee1 = 132+(fah1- 240 )*0.70;
            }
            else {
                fee1 = 342 + (fah1- 540 )*0.95;
            }
            float fee2 = (float) (Math.round(fee1 * 100) / 100.0);
            fee.setText(fee2 + "元");
        }
        catch(NumberFormatException e) {
            getConsumption.setText( "输入错误");
            fee.setText( "");
        }
    }
}

```

(2) 实验流程图



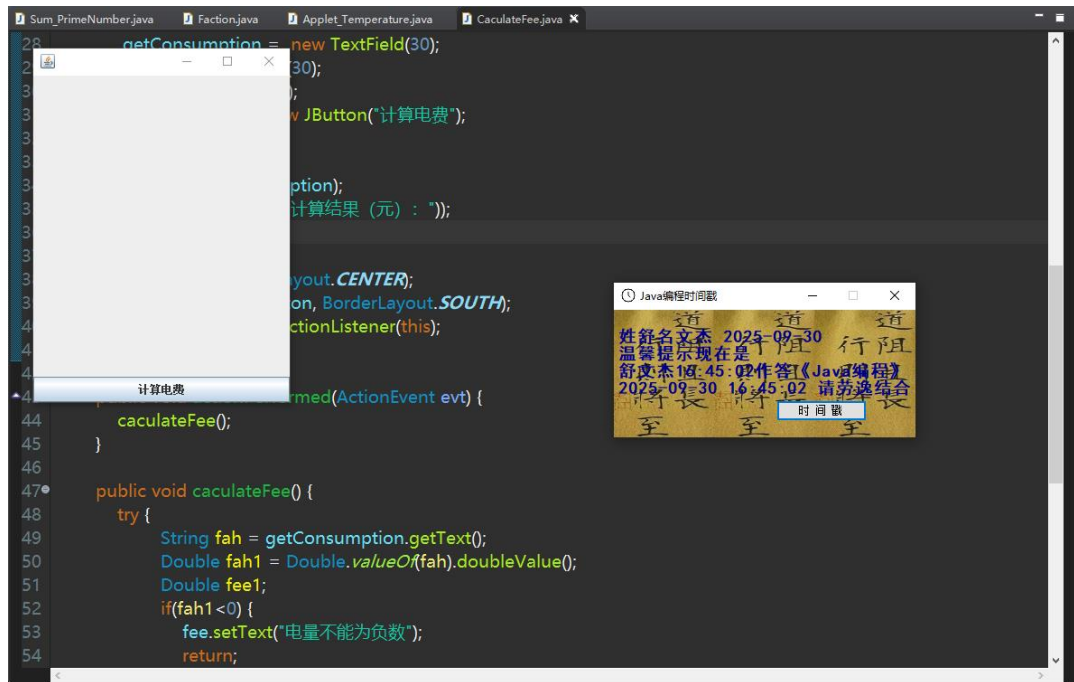
(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验思路：我们同样使用 Applet 程序进行编写，因此，我们上一个程序的代码便可以最大程度地复用，我们可以直接将大部分代码框架直接搬过来，需要修改的部分在于我们使用多次 if-else 语句处理阶梯的水费问题。但是我们也可以使用更加现代的 swing 类来写这个小程序，因此，我们采用了后者。对于保留两位小数，我们可以参照上次的程序的方案：

```
float fah3 = (float) (Math.round(fah2 * 100) / 100);
```

便可以轻松完成。

1) 使用 swing 编写电费计算小程序并运行



?, 为什么我的小程序只有一个按钮, 我的界面显示怎么不见了? 查阅相关资料发现, 原来我的 JFrame 默认使用 BorderLayout 布局, 其 CENTER 位置只能显示最后一个添加的组件。你先往 JFrame 里添加了多个组件 (会互相覆盖), 最后添加的 jp 本身是空的 (没有任何组件), 所以界面就变成了空白。

2) 修改我们的代码后, 我们直接在 jp 中直接添加组件

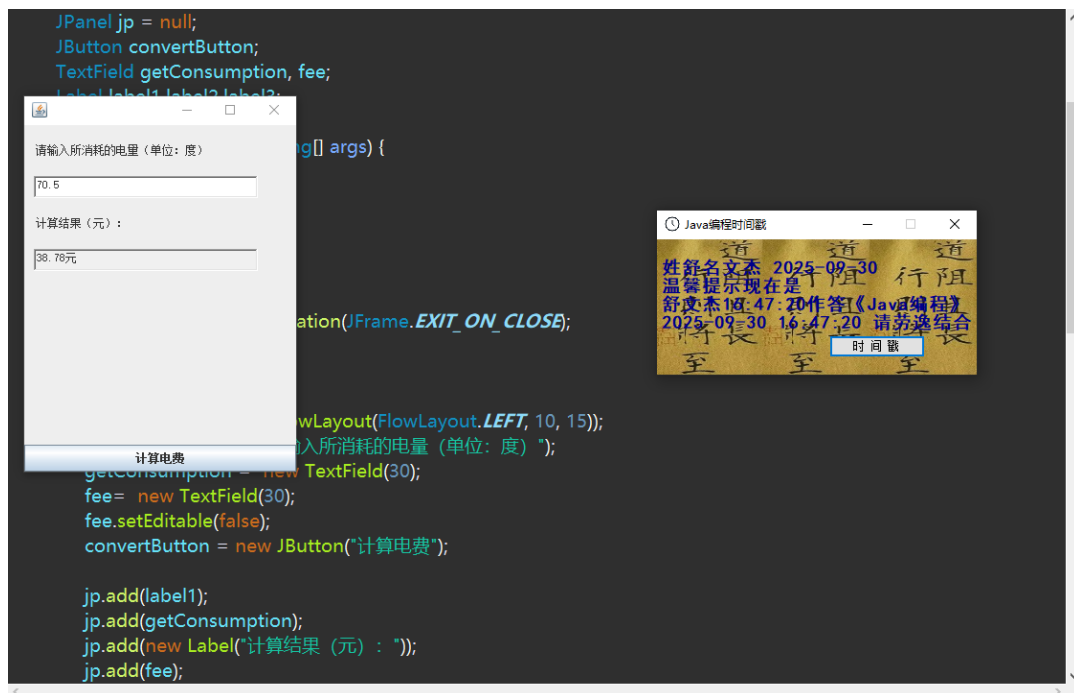


图 9 实验结果截图

实验结果符合预期, 实验成功。但是, 我同时还发现一个问题: (当我们的结

果的第二位小数为 0，它会自动省略第二位小数)

因此，由第一次

3) 考虑异常处理，这里我们要额外考虑负数的情况

```
if(fah1<0) {
    fee.setText("电量不能为负数");
    return;
}
```

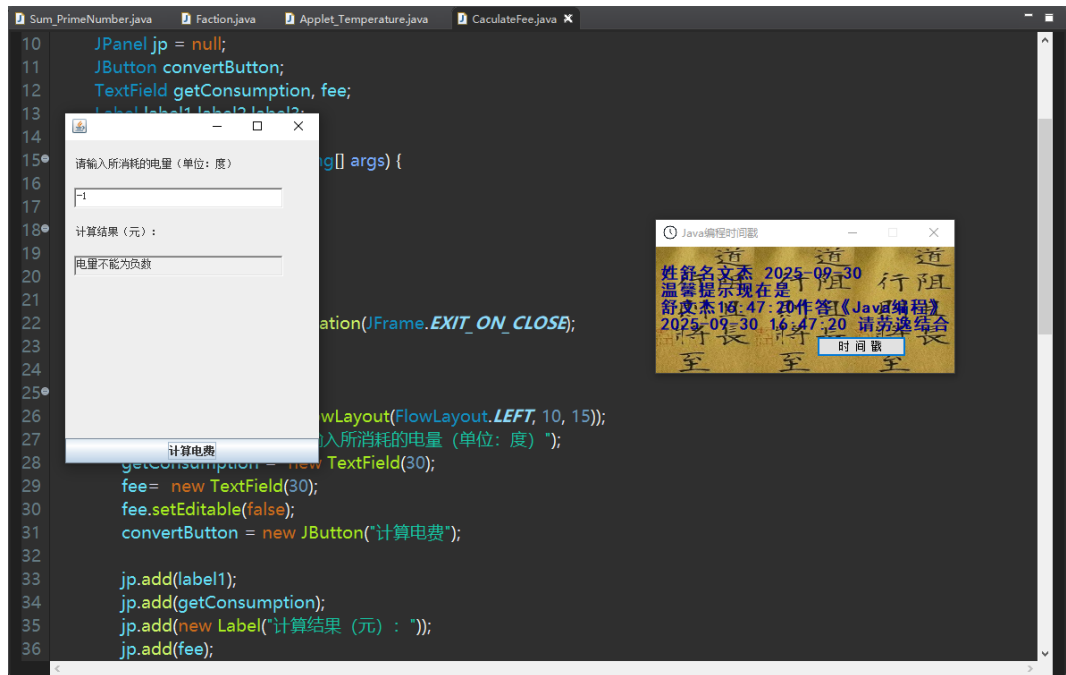


图 10 异常处理结果截图

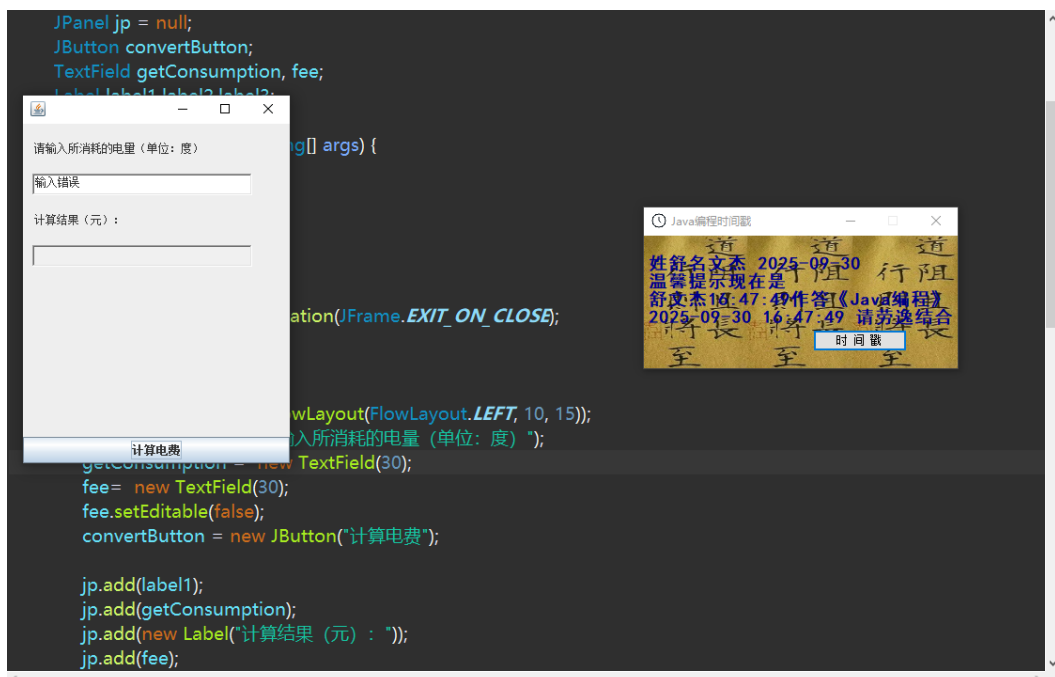


图 11 异常处理结果截图

综上，该程序的功能以及完美实现。

【实验心得】

一、Eclipse 从 “陌生工具” 变成 “得力助手”

一开始看到实验目的里 “熟悉 Eclipse 的使用”，我还觉得就是随便点一点界面，没想到真上手才发现全是细节。写质数求和程序时，我随手写了 `isPrime` 方法，没加 `static` 就往 `main` 里调，结果 Eclipse 立马飘红，报错提示 “静态方法不能调用非静态方法”——以前光记这句话没感觉，这次看着红色波浪线，才实实在在懂了 “`static`” 的约束。后来做阶梯电费的 Swing 界面，我把组件一个个往 `JFrame` 里加，运行后界面居然一片空白！我盯着屏幕翻来覆去看代码，最后还是靠 Eclipse 的 “检查语法” 功能，才发现 `JFrame` 默认的 `BorderLayout` 布局里，`CENTER` 位置只能显示最后一个组件，前面加的全被覆盖了。这两次踩坑下来，我再也不是只会点 “运行” 的新手了，现在看报错提示、查布局问题都快多了，Eclipse 总算用顺手了。

四、GUI 开发从 “完全不懂” 到 “能做个小界面”

以前看别人做的图形界面觉得好复杂，这次自己动手试了，才慢慢摸出门道。做温度转换 Applet 时，我对着教程一步步来：在 `init` 方法里加 `Label`（显示 “摄氏温度”）、`TextField`（让用户输入）、`Button`（转换按钮），还得给按钮绑 `ActionListener`——一开始我忘了绑监听，点按钮没反应，后来才知道 “得告诉程序 ‘点按钮要做什么’”。等这些组件都弄好，输入数字点转换能出结果时，我还挺有成就感的。后来做阶梯电费改用 Swing 的 `JFrame`，又踩了布局的坑：直接往 `JFrame` 里加输入框、按钮，结果界面只剩最后一个组件。我查了资料才知道，`BorderLayout` 的五个位置里，`CENTER` 会覆盖前面的组件，得用 `JPanel` 把组件包起来再加进去。现在我能做出 “输入电量 - 点按钮 - 出电费” 的小界面了，也

明白 GUI 不是“堆组件”，而是“组件初始化 - 绑事件 - 写响应逻辑”的一套流程。

二、终于明白“好程序不光能跑，还得抗造”

以前写代码总想着“只要正常输入能出结果就行”，这次实验才知道，还得考虑“万一用户瞎输入怎么办”。比如温度转换和阶梯电费，一开始我没处理异常，用户输个字母进去，程序直接崩了，报错“`NumberFormatException`”。后来学了 `try-catch`，把输入转换的代码包在 `try` 里，`catch` 住这个异常，再提示“输入错误”，程序就不会崩了。做阶梯电费时，我还特意加了“如果电量是负数，就提示‘电量不能为负数’”——毕竟现实里没人用负电量，总不能让程序算出负的电费吧？现在我写代码会多问自己一句：“如果输入不对怎么办？”“有没有没考虑到的情况？”原来好程序不光要实现核心功能，还得能应对各种意外，这样用户用起来才放心。

三、遇到问题不再“等答案”，自己找办法更踏实

这次实验遇到的问题，几乎都是我自己查资料解决的。比如主类没加 `public` 报错、`static` 缺失、Swing 布局空白，一开始我也慌，不知道怎么弄，但后来我先看 Eclipse 的报错提示，提示看不懂就搜“Java 主类不加 `public` 报错”“JFrame 界面空白”，翻 Java 文档、技术博客，慢慢就找到解决办法了。印象最深的是阶梯电费的小数显示问题：算出来的电费有时候会少个末尾的 0（比如该显示 123.40，结果只显 123.4），我搜了好久，才知道用 `Math.round` 把结果乘 100 再除以 100，再结合格式化输出就能解决。还有为了搞懂 Applet 为啥会自动绘图，我特意查了 Applet 的生命周期，知道 `init` 之后会调用 `paint` 方法，才算明白原理。现在我觉得，遇到问题不可怕，关键是“先分析哪里错了，再找资料解决”，这种自己摸索的过程，比直接拿答案记得牢多了。